YOLO11n Cross-Platform Benchmark

Alvin Le, Marcelo Perez Masilungan Jr., Thomas J. Orr, Daniel Romero, Victor Franco Venegas
California State Polytechnic University, Pomona

Abstract

This report evaluates YOLO11n inference performance across three platforms: Raspberry Pi 5, Windows PC with NVIDIA GPU, and MacBook Air M2. Using standardized benchmarking, latency, throughput, temperature, and power were measured. The MacBook Air achieved the best latency at 55.70 ms, while the Raspberry Pi showed the slowest performance at 419.60 ms. These results demonstrate that architecture, memory organization, and access to acceleration hardware strongly affect real-time computer vision performance. The comparison highlights the tradeoff between low-power embedded systems, conventional desktop GPU systems, and unified-memory architectures for lightweight object detection workloads.

Context

Object detection is one of the most important tasks in computer vision because it allows a system not only to classify what is present in an image, but also to determine where those objects are located. Applications such as robotics, autonomous navigation, industrial inspection, and smart surveillance depend on fast and accurate object detection in order to operate effectively in real time.

YOLO, which stands for "You Only Look Once," is a family of object detection models designed for high-speed inference. Unlike region-based methods that break detection into multiple stages, YOLO performs object localization and classification in a single forward pass through the network. This design greatly improves speed and makes the model suitable for systems where latency is a major concern.

YOLO11n is a lightweight nano variant intended for fast inference and lower compute demand. Because of its smaller size, it is useful for benchmarking across a range of hardware targets, from embedded devices to laptops and desktop systems. Even so, actual performance depends heavily on the platform running the model. Differences in CPU architecture, GPU access, thermal behavior, and memory organization can all influence inference time and throughput.

This project examines these effects by comparing YOLO11n on three very different platforms: a Raspberry Pi 5, a Windows PC with an NVIDIA GPU, and a MacBook Air M2. Together, these systems provide a useful contrast between embedded CPU-only inference, discrete-GPU acceleration, and Apple Silicon unified-memory execution.

Objective

The primary goal of this project is to compare the practical inference performance of YOLO11n across different computing architectures. Rather than focusing only on raw speed, the study also considers how system design influences stability, thermal behavior, and real-time usability.

The objectives of this benchmark are:

- Measure inference latency on each platform under repeated runs
- Compare throughput in frames per second across systems
- Examine how memory design and hardware acceleration affect results
- Evaluate the suitability of each device for real-time object detection
- Identify tradeoffs between efficiency, portability, and performance

A secondary objective is to provide a hardware-aware interpretation of the results. This is important because modern machine learning workloads are shaped not only by model design, but also by the way compute resources and memory are arranged in the system.

Methods

Each platform executed a Python benchmark script that repeatedly ran YOLO11n inference and recorded summary performance metrics. The same general procedure was followed across systems so that the comparison would reflect hardware differences rather than major methodological changes.

For each run, the benchmark measured per-inference latency and used that value to estimate throughput in frames per second. Additional system metrics were collected when available. These included CPU utilization, RAM usage, temperature, and power draw. Because hardware interfaces differ by operating system and platform, not every metric was available on every system. Even so, latency and throughput were measured consistently and remain the main basis for comparison.

Summary statistics included:

- Average latency
- Median latency
- Standard deviation
- Minimum and maximum latency
- Throughput in frames per second

The Raspberry Pi 5 executed 50 runs in a CPU-only configuration. The Windows PC executed 50 runs with an NVIDIA GPU available. The MacBook Air M2 also executed 50 runs using Apple Silicon in a CPU-only benchmark configuration. These run counts were sufficient to capture representative behavior while keeping test duration manageable.

Platforms

The three platforms used in this benchmark represent substantially different hardware classes.

Raspberry Pi 5: The Raspberry Pi 5 is a compact embedded platform using an ARM Cortex-A76 CPU. In this benchmark it

Metric	RPi 5	Win. PC	Mac M2
Avg Latency (ms)	449.52	72.72	55.70
Median (ms)	443.26	71.00	55.61
Std Dev (ms)	20.12	5.86	3.31
Min/Max (ms)	421.34/499.08	68.04/98.86	43.35/74.84
Throughput (FPS)	2.22	13.8	18.01
Avg Temp	71.4°C	52.2°C	N/A
Avg Power	4.12 W	15.5 W	N/A
Architecture	ARM A76	x86+ NVIDIA	ARM64 Unified
GPU	None	NVIDIA	None

Table 1: Summary benchmark comparison across all three platforms.

operated without a discrete GPU or dedicated neural accelerator. This makes it a good example of edge deployment on constrained hardware, where power efficiency and cost are important but compute resources are limited.

Windows PC with NVIDIA GPU: The desktop platform combines a conventional x86-64 CPU with a discrete NVIDIA GPU. This setup represents a more traditional high-performance machine learning environment where acceleration is available, but where GPU memory remains separate from system memory. That distinction can introduce overhead when input data must move between memory regions.

MacBook Air M2: The MacBook Air M2 uses Apple Silicon with an ARM64 architecture and unified memory. In unified-memory systems, major compute resources share access to the same memory pool. This can reduce transfer overhead and improve efficiency, especially for smaller models where memory movement becomes a meaningful portion of total inference time.

Mac Environment Setup

The MacBook Air required a dedicated Python environment for testing. The setup used for the benchmark is shown below.

```
curl -O https://repo.anaconda.com/miniconda/
Miniconda3-latest-MacOSX-arm64.sh
bash Miniconda3-latest-MacOSX-arm64.sh

conda create -n yolo-bench python=3.11 -y
conda activate yolo-bench
pip install ultralytics psutil

python benchmark.py
```

Results

Performance Comparison

The results show a clear performance hierarchy across the three systems. The MacBook Air M2 achieved the lowest average latency and the highest throughput. The Windows PC placed second, offering strong acceleration but not matching the MacBook's latency. The Raspberry Pi 5 was substantially slower than both, producing frame rates far below real-time expectations.

In addition to average latency, variability also matters. The MacBook Air M2 showed lower variation than the Windows PC, suggesting a more stable execution pattern across repeated runs. The Windows system, while fast, displayed a wider range of latencies, which may reflect workload scheduling or data transfer effects associated with discrete GPU use.

The following subsections move platform by platform so that the benchmark output, dashboard views, and interpretation.

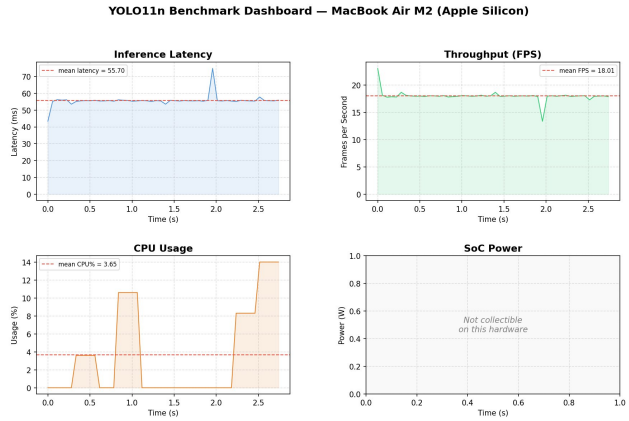


Figure 1: Terminal output from benchmark on MacBook Air M2 (50 runs).

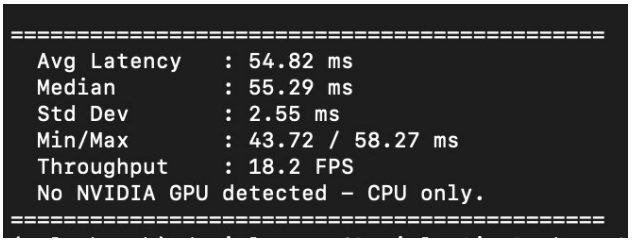


Figure 2: MacBook Air M2 benchmark dashboard showing latency and available system metrics.

MacBook Air M2 Results

The MacBook Air M2 produced the strongest overall benchmark result, with an average latency of 55.70 ms and throughput of 18.01 FPS. This performance is especially notable because the system achieved it without relying on a discrete GPU. The results indicate that Apple Silicon's architecture is highly effective for lightweight inference workloads.

The raw benchmark output is shown first in Figure 1. This view provides the direct recorded results from the repeated runs and establishes the low-latency baseline observed on the MacBook platform.

The terminal summary is reinforced by the dashboard view in Figure 2, which makes the latency behavior and associated system metrics easier to interpret visually. Together, these first two figures show that the MacBook maintained strong performance across the full benchmark rather than achieving a good average only because of a few unusually fast runs.

To further connect performance with resource behavior, Figure 3 shows CPU and RAM usage during inference. This figure helps explain why the platform remained stable over repeated runs and supports the conclusion that YOLO11n is a comfortable workload for the system.

The dashboard and terminal output suggest stable performance across repeated runs. Low variability indicates that the platform handled repeated inference without significant spikes in resource contention or throttling. CPU and RAM usage also remained well controlled, supporting the idea that YOLO11n is a comfortable workload for the platform.

One of the main reasons for this performance is the unified-memory design of the M2. For smaller models, moving data ef-

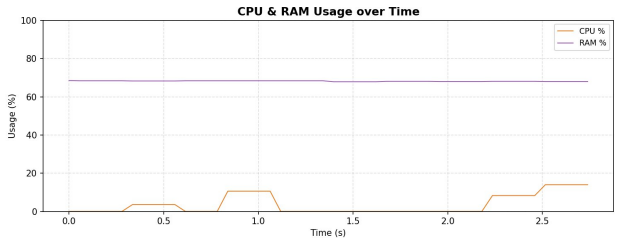


Figure 3: CPU and RAM usage during MacBook Air M2 inference.

ficiently can matter as much as raw compute capability. Because the MacBook avoids the overhead of transferring data between separate CPU and GPU memory spaces, it can perform inference with reduced latency and better consistency.

Windows PC Results

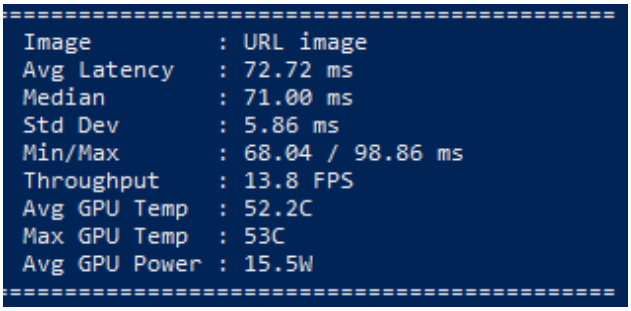
The Windows PC delivered solid performance, reaching an average latency of 72.72 ms and throughput of 13.8 FPS. This confirms that the system benefits significantly from access to a discrete NVIDIA GPU. Compared to the Raspberry Pi 5, the improvement is dramatic, showing how much acceleration hardware can help with modern deep learning inference.

Figure 4 presents the terminal output from the Windows benchmark. This figure shows the direct numerical results from the repeated runs and establishes that the platform is clearly faster than the embedded system while still trailing the MacBook Air M2 in average latency.

Figure 5 then summarizes the same benchmark session in dashboard form. This figure should be included without aggressive trimming so the subplot titles remain visible.

At the same time, the Windows PC did not outperform the MacBook Air M2 in this benchmark. This result is important because it shows that a discrete GPU does not automatically guarantee the best latency for every workload. YOLO11n is relatively lightweight, so transfer overhead between system RAM and GPU VRAM may become a more noticeable fraction of the total execution time.

The benchmark also showed more variation in latency than the MacBook. That higher spread may reflect fluctuations in GPU scheduling, driver behavior, or memory transfer timing. Even with those effects, the Windows PC remains clearly suitable for near-real-time workloads and offers much stronger performance than a CPU-only embedded platform.



```
================================================
Image          : URL image
Avg Latency    : 72.72 ms
Median         : 71.00 ms
Std Dev        : 5.86 ms
Min/Max        : 68.04 / 98.86 ms
Throughput     : 13.8 FPS
Avg GPU Temp   : 52.2C
Max GPU Temp   : 53C
Avg GPU Power  : 15.5W
================================================
```

Figure 4: Windows PC terminal output from the benchmark.

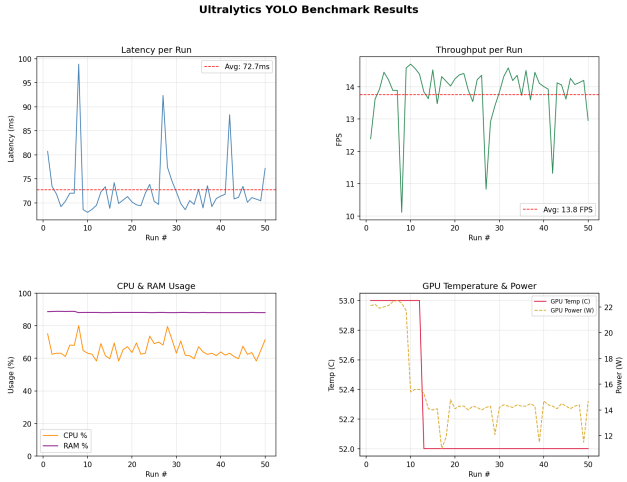
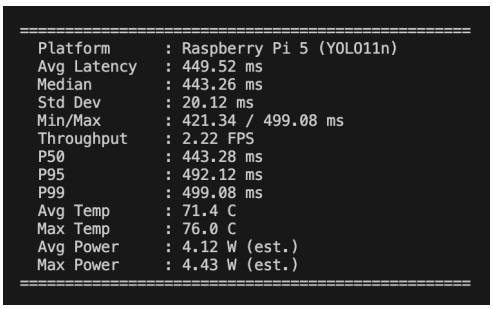


Figure 5: Windows benchmark results dashboard.

Raspberry Pi 5 Results

The Raspberry Pi 5 recorded the slowest performance by a wide margin, with an average latency of 449.52 ms and throughput of only 2.22 FPS. This result is consistent with expectations for an embedded CPU-only system running neural network inference without dedicated acceleration.

The first Raspberry Pi figure, shown in Figure 6, presents the terminal output from the benchmark script. This output makes the large latency difference immediately visible and provides the direct benchmark summary for the embedded platform.



```
================================================
Platform       : Raspberry Pi 5 (YOLO11n)
Avg Latency    : 449.52 ms
Median         : 443.26 ms
Std Dev        : 20.12 ms
Min/Max        : 421.34 / 499.08 ms
Throughput     : 2.22 FPS
P50            : 443.28 ms
P95            : 492.12 ms
P99            : 499.08 ms
Avg Temp       : 71.4 C
Max Temp       : 76.0 C
Avg Power      : 4.12 W (est.)
Max Power      : 4.43 W (est.)
================================================
```

Figure 6: Terminal output from `benchmark.py` on Raspberry Pi 5.

Figure 7 expands on those raw results by showing the benchmark dashboard, including latency, throughput, CPU usage, temperature, and power behavior. Placing this figure immediately after the terminal output helps connect the numerical summary to the broader system-level limitations of the device.

Although the Raspberry Pi was able to complete the workload

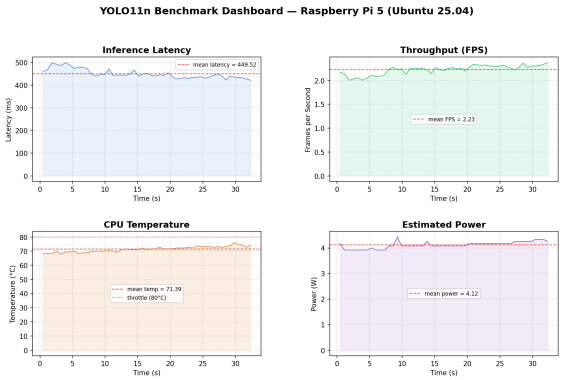


Figure 7: Benchmark dashboard for Raspberry Pi 5 showing latency, throughput, CPU usage, temperature, and power.

successfully, its frame rate is well below the threshold needed for most real-time vision tasks. In practical terms, this means that while the model is deployable on the platform, responsiveness would be limited unless the application can tolerate long delays or infrequent updates.

Thermal and power data further emphasize the system's constraints. The recorded temperature suggests sustained load during inference, and the estimated power draw remains low in absolute terms, reflecting the device's embedded design. The Raspberry Pi therefore offers an attractive low-cost and low-power option, but only when performance requirements are modest or external acceleration is added.

Discussion

The benchmark results demonstrate that hardware architecture has a major effect on YOLO11n inference performance. Although all three systems were capable of running the model, the quality of that performance differed substantially.

The Raspberry Pi 5 represents the lower end of the performance range. Its results show that CPU-only inference on an embedded device is possible, but not well suited for real-time object detection when using an unaccelerated model. The main limitations are limited compute resources, thermal constraints, and the absence of specialized acceleration hardware. For edge applications, this means that either a smaller or quantized model, or an attached accelerator, would likely be necessary.

The Windows PC occupies the middle of the comparison. Its discrete NVIDIA GPU provides substantial acceleration and makes the workload practical for real use. However, the system also illustrates the cost of separate memory domains. When a model is relatively small, the overhead of moving data into and out of GPU memory can reduce the advantage of raw hardware throughput. This helps explain why the Windows PC, despite having a dedicated GPU, did not achieve the lowest latency in the benchmark.

The MacBook Air M2 produced the best results overall. Its performance suggests that unified memory can be highly beneficial for lightweight inference workloads. By reducing data transfer overhead and integrating compute resources more tightly, the platform achieved both lower latency and lower variance. This does not mean that unified-memory systems always outperform discrete GPUs, but it does show that architecture matters greatly when workload size is modest.

A key lesson from the benchmark is that model deployment decisions should not be based only on peak hardware specifications. Instead, they should consider the specific workload, the movement of data through the system, and the operational requirements of the application. For YOLO11n, the MacBook Air M2 provided the best balance of speed and efficiency, the Windows PC provided strong but somewhat less efficient acceleration, and the Raspberry Pi 5 highlighted the limitations of embedded inference without dedicated support.

Conclusion

This study compared YOLO11n inference performance on three platforms with very different architectural designs: an embedded ARM CPU system, a desktop x86-64 plus NVIDIA GPU system, and an Apple Silicon unified-memory system. The results showed a clear separation in capability, with the MacBook Air M2 producing the lowest latency, the Windows PC delivering solid GPU-accelerated performance, and the Raspberry Pi 5 trailing substantially behind.

These findings reinforce the idea that hardware architecture strongly influences modern AI inference performance. For lightweight models, efficient memory organization can be just as important as raw compute power. The MacBook Air M2 benefited from this effect, while the Windows PC showed that discrete GPU overhead can still matter. The Raspberry Pi, meanwhile, demonstrated that low-cost embedded deployment remains possible, but often at the expense of real-time responsiveness.

Future work could extend this benchmark by testing quantized versions of YOLO11n, exploring additional input sizes, or integrating dedicated accelerators such as a Coral TPU or similar device for the Raspberry Pi. Such experiments would provide a more complete picture of how lightweight object detection models scale across edge and desktop hardware.

References

1. Ultralytics YOLO Documentation: `https://docs.ultralytics.com`